lista ava/comparções.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

long long int comparacoes = 0;
long long int trocas = 0;
#define subvetores 32

int* arranjoCrescente(int n) {
    int* v = (int*) malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) v[i] = i;
    return v;
}

int* arranjoDecrescente(int n) {
    int* v = (int*) malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) v[i] = n - 1 - i;
    return v;
}

int* arranjoAleatorio(int n) {
    int* v = (int*) malloc(n * sizeof(int));
    srand(time(NULL));
    for (int i = 0; i < n; i++) v[i] = rand();
    return v;
}

void insertionSort(int p, int r, int v[]) {
    for (int i = p + 1; i < r; i++) {
        int key = v[i];
        int j = i - 1;
        while (j >= p && (++comparacoes && v[j] > key)) {
            v[j + 1] = v[j];
            j--;
            trocas++;
        }
        v[j + 1] = key;
    }
}

void intercala(int p, int q, int r, int v[]) {
    int n1 = q - p;
    int n2 = r - q;
    int* L = (int*) malloc(n1 * sizeof(int));
    int* R = (int*) malloc(n2 * sizeof(int));

    for (int i = 0; i < n1; i++) L[i] = v[p + i];
    for (int i = 0; i < n2; i++) R[i] = v[q + i];

    int i = 0, j = 0, k = p;
    while (i < n1 && j < n2) {
```

```c
        comparacoes++;
        if (L[i] <= R[j]) v[k++] = L[i++];
        else v[k++] = R[j++];
        trocas++;
    }
    while (i < n1) v[k++] = L[i++];
    while (j < n2) v[k++] = R[j++];

    free(L);
    free(R);
}

void mergesort(int p, int r, int v[]) {
    if (r - p <= subvetores) {
        insertionSort(p, r, v);
        return;
    }
    if (p < r-1) {
        int q = (p + r) / 2;
        mergesort(p, q, v);
        mergesort(q, r, v);
        intercala(p, q, r, v);
    }
}

void quicksort(int esq, int dir, int v[]) {
    if (dir - esq + 1 <= subvetores) {
        insertionSort(esq, dir + 1, v);
        return;
    }

    int i = esq, j = dir;
    int pivo = v[esq];

    while (i <= j) {
        while ((++comparacoes && v[i] < pivo)) i++;
        while ((++comparacoes && v[j] > pivo)) j--;
        if (i <= j) {
            int aux = v[i]; v[i] = v[j]; v[j] = aux;
            trocas++;
            i++; j--;
        }
    }
    if (esq < j) quicksort(esq, j, v);
    if (i < dir) quicksort(i, dir, v);
}


void testar(int n, char* desc, int* (*gerador)(int), void (*algoritmo)(int,int,int[])) {
    int* v = gerador(n);
    comparacoes = 0;
    trocas = 0;
    clock_t inicio = clock();
```

```c

    algoritmo(0, n-1, v);

    clock_t fim = clock();
    double tempo = (double)(fim - inicio) / CLOCKS_PER_SEC;

    printf("%s | Tamanho: %d | Comparações: %lld | Trocas: %lld | Tempo: %.5f s\n",
           desc, n, comparacoes, trocas, tempo);

    free(v);
}

int main() {
    int tamanhos[] = {10000, 100000, 1000000};
    int numTamanhos = sizeof(tamanhos) / sizeof(tamanhos[0]);

    printf("=== Merge Sort  ===\n");
    for (int i = 0; i < numTamanhos; i++) {
        int n = tamanhos[i];
        testar(n, "Crescente", arranjoCrescente, mergesort);
        testar(n, "Decrescente", arranjoDecrescente, mergesort);
        testar(n, "Aleatorio", arranjoAleatorio, mergesort);
        printf("----------------------------------------\n");
    }

    printf("=== Quick Sort ===\n");
    for (int i = 0; i < numTamanhos; i++) {
        int n = tamanhos[i];
        testar(n, "Crescente", arranjoCrescente, quicksort);
        testar(n, "Decrescente", arranjoDecrescente, quicksort);
        testar(n, "Aleatorio", arranjoAleatorio, quicksort);
        printf("----------------------------------------\n");
    }

    return 0;
}
```